

Zero to Hero Study Handbook: Multi-Agent System using Local Ollama

Module 1: Foundations & Architecture

1.1 What this project does

This repository implements a local, production-style multi-agent question-answering system over a small operational knowledge base.

It is built around four specialized agents orchestrated in sequence: - **RouterAgent** classifies intent and severity from a question. - **RetrieverAgent** retrieves top-k relevant local documents. - **PlannerAgent** drafts an answer (LLM-first, deterministic fallback on error/timeout/empty output). - **ReviewerAgent** appends operational guardrails and source cross-check guidance.

Main use cases reflected in code and data: - Incident/release/support/governance Q&A over local .md docs in `data/knowledge/`. - Baseline vs multi-agent evaluation over tasks in `data/eval/tasks.json`. - Telemetry and report generation into `artifacts/`. - Notebook-first learning path in `notebooks/`.

1.2 Core paradigms and patterns used (with definitions)

1. **Orchestrator Pattern** Definition: A central coordinator calls specialized components in a fixed flow. Where used: `MultiAgentCoordinator.run()` in `src/multi_agent_system/orchestration/coordinator.py`.
2. **Specialized-Agent Decomposition** Definition: Split one complex workflow into small role-focused agents. Where used: `RouterAgent`, `RetrieverAgent`, `PlannerAgent`, `ReviewerAgent` under `src/multi_agent_system/agents/`.
3. **Schema-First Design (Pydantic + Dataclass)** Definition: Use typed models to enforce input/output structure. Where used: `KnowledgeDoc`, `TaskExample`, `ChatResult`, `AgentRoute`, `AgentRunResult`, `EvalPrediction`, `EvalSummary`, `TraceSpanRecord` in `src/multi_agent_system/schemas.py`.
4. **Deterministic Fallback for Reliability** Definition: If LLM calls fail or return empty text, return predictable fallback output. Where used: `PlannerAgent.plan()` and `PlannerAgent.baseline()` in `src/multi_agent_system/agents/planner.py`; reviewer fallback in `src/multi_agent_system/agents/reviewer.py`.
5. **Context-Manager Telemetry Instrumentation** Definition: Wrap stage execution in context managers that emit JSONL spans with latency/status. Where used: `JsonlTelemetryTracer.span()` in `src/multi_agent_system/telemetry/tracer.py`.
6. **Lexical Retrieval Pipeline** Definition: Score docs by token overlap between query terms and document terms. Where used: `retrieve_docs()`

in src/multi_agent_system/tools/knowledge_base.py.

1.3 Architecture overview and interactions

Main runtime path (CLI): multi-agent-system run-all -> run_all(settings)
-> run_demo(...) + run_evaluation(...) -> report + telemetry summaries
+ run summary artifact.

ASCII architecture diagram (main flow):

```
[User CLI Command]
|
v
[Type app: src/multi_agent_system/cli.py]
  callback() -> get_settings() -> configure_logging() -> ensure_dirs()
  |
  v
[pipeline.py]
  _build_coordinator(settings, tracer)
  |
  +--> load_knowledge_docs(data/knowledge/*.md)
  +--> AsyncOllamaGateway(host)
  +--> RouterAgent
  +--> RetrieverAgent(top_k)
  +--> PlannerAgent(settings, gateway)
  +--> ReviewerAgent
  |
  v
[MultiAgentCoordinator.run(question, trace_id)]
  route -> retrieve -> baseline_answer -> plan_answer -> review_answer
  |           |           |           |           |
  +-----+-----+-----+-----+-----+
                                each stage wrapped in tracer.span(...)
  |
  v
[AgentRunResult]
  trace_id, route, retrieved, baseline_answer, plan_answer, final_answer,
  planner_fallback_used, reviewer_fallback_used, total_latency_ms
  |
  +--> run_demo(): save_demo_runs() -> artifacts/runs/demo_runs.json
  |
  +--> run_evaluation(): evaluate() -> save_predictions() + save_summary()
  |                       -> summarize_traces() -> render_report()
  |
  v
[artifacts/*]
  evals/predictions.csv
```

```

evals/summary.json
telemetry/traces.jsonl
telemetry/summary.json
reports/multi_agent_system_report.md
run_summary.json

```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Project metadata, dependencies, entrypoint	[project.scripts] Python multi-agent-system = "multi_agent_system_ollama:app"	=3.12.10,<3.13; deps include pydantic, typer, loguru
.env.example	Runtime environment template	N/A	OLLAMA_HOST, PLANNER_MODEL, REVIEWER_MODEL, RETRIEVAL_TOP_K, GENERATION_*, artifact/data paths
README.md	Usage overview and command guide	N/A	Setup commands and run commands
src/multi_agent_system/cli.py	CLI entrypoint and command wiring	app, check_models_cmd, run_demo_cmd, evaluate_cmd, run_all_cmd, summarize_telemetry_cmd	Default demo questions in run_demo_cmd
src/multi_agent_system/settings.py	System settings and path resolution	Settings, get_settings, ensure_dirs	Pydantic BaseSettings; path properties like predictions_file, summary_file, traces_file
src/multi_agent_system/pipeline.py	System pipeline orchestration of demo/eval/report workflows	_build_coordinator, run_demo, run_evaluation, run_all	demo_questions list, reset_traces behavior

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/multi_agent_system/orchestration/multi_agent_executor.py	System orchestration multi-agent execution	MultiAgentCoordinator	Stage order and per-stage telemetry span names
src/multi_agent_system/agents/router.py	System/agents/ routing	RouterAgent.route	Keyword rules for incident, release, support, governance, fallback general
src/multi_agent_system/agents/retriever.py	System/agents/ retriever wrapper	RetrieverAgent.retrieve top_k	Lexical retriever with configurable top_k
src/multi_agent_system/tools/knowledge_base.py	System/tools/ knowledge loading and lexical scoring	KnowledgeBase.py, _tokenize, load_knowledge_docs, retrieve_docs	SUPPORTED_SUFFIXES = {".md", ".txt"}
src/multi_agent_system/agents/planner.py	System/agents/ generation + deterministic fallback	PlannerAgent.plan, PlannerAgent.base_line, _context_block, _fallback	PLANNER_SYSTEM_PROMPT; line generation settings from Settings
src/multi_agent_system/agents/reviewer.py	System/agents/ review and source cross-check text append	ReviewerAgent.review	Reviewer fallback for empty draft
src/multi_agent_system/Ollama_client.py	System/Ollama API adapter	AsyncOllamaGateway, ensure_required_models, chat	First, model names, models, asyncio.wait_for; maps response metadata into ChatResult
src/multi_agent_system/schemas.py	System/schemas models	KnowledgeDoc, TaskExample, ChatResult, AgentRoute, AgentRunResult, TraceSpanRecord, EvalPrediction, EvalSummary	Strong typing with extra="forbid" on models

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/multi_agent_system/eval/evaluator.py	System-level evaluation and metric aggregation	load_eval_tasks, keyword_recall, evaluate	Metrics: recall, gain, retrieval hit rate, source citation rate, fallback rates
src/multi_agent_system/telemetry/tracer.py	System-level telemetry and telemetry aggregation	facade, TelemetryTracer, summarize_traces	Spans, mean/p50/p95/max latency computation
src/multi_agent_system/reports/reporting.py	Artifact reporting, persistence and markdown report rendering	reporting_options, save_summary, save_demo_runs, render_report	Jinja2, _REPORT_TEMPLATE
scripts/run_pipeline.py	Snippet entry to run entire pipeline and print payload	_main	Calls run_all(settings)
scripts/generate_notebooks.py	Rootbook typically creates tutorial notebooks	main, _write	Notebook order 01 to 05
scripts/execute_notebooks.py	Executes notebooks sequentially	execute, main	NOTEBOOKS list and NotebookClient(timeout=1800)
data/knowledge/*.txt	Local knowledge corpus used by retriever	N/A	Incident, release, support, governance, model-ops facts
data/eval/tasks.json	Evaluation task definitions	N/A	Keys: task_id, question, reference_answer, expected_source, required_keywords
artifacts/	Persisted outputs from runs/evaluation	N/A	evals/, telemetry/, reports/, runs/, run_summary.json
tests/*.py	Behavioral contracts and regression checks	Router/retrieval/metrics tests	Encodes expected behavior for key helpers

Module 3: Core Execution Flows

3.1 Flow A: CLI bootstrapping and command dispatch

Entrypoint chain 1. Packaging registers `multi-agent-system` command in `pyproject.toml`. 2. This points to `src/multi_agent_system/cli.py:app` (Typer application). 3. `@app.callback()` runs for all commands: - `settings` = `get_settings()` - `configure_logging()` - `settings.ensure_dirs()`

Available commands in cli.py - `check-models` - `run-demo` - `evaluate` - `run-all` - `summarize-telemetry`

3.2 Flow B: Per-question multi-agent run (`MultiAgentCoordinator.run`)

Source: `src/multi_agent_system/orchestration/coordinator.py`

Step-by-step: 1. Start timer (`perf_counter`). 2. Span `route`: call `RouterAgent.route(question) -> AgentRoute`. 3. Span `retrieve`: call `RetrieverAgent.retrieve(question) -> list[RetrievedDoc]`. 4. Span `baseline_answer`: call `PlannerAgent.baseline(question)`. 5. Span `plan_answer`: call `PlannerAgent.plan(question, route, retrieved, trace_id)`. 6. Span `review_answer`: call `ReviewerAgent.review(question, draft_answer=plan, retrieved=retrieved)`. 7. Compute total latency and return `AgentRunResult`.

Short fragment (real control order):

```
with self._tracer.span(trace_id, "route", {"question_chars": len(question)}):
    route = self._router.route(question)

with self._tracer.span(trace_id, "retrieve", {"top_k": self._retriever.top_k}):
    retrieved = self._retriever.retrieve(question)

with self._tracer.span(trace_id, "baseline_answer"):
    baseline = await self._planner.baseline(question)

with self._tracer.span(trace_id, "plan_answer", {"intent": route.intent}):
    plan = await self._planner.plan(question=question, route=route, retrieved=retrieved, trace_id=trace_id)

with self._tracer.span(trace_id, "review_answer"):
    final = await self._reviewer.review(question=question, draft_answer=plan, retrieved=retrieved, trace_id=trace_id)
```

3.3 Flow C: Routing + retrieval internals

Router output shape (`AgentRoute`) Defined in `schemas.py`:

```
{
    "intent": "incident|release|support|governance|general",
    "severity": "S0|S1|S2|S3|NA",
}
```

```

    "reason": "string"
}

```

Rule highlights from RouterAgent.route(): - Contains incident|outage|latency|rollback
-> intent incident. - Contains outage|rollback -> severity S1; otherwise
incident severity S2. - Contains release|canary|deploy -> release/NA.
- Contains support|sla|ticket|escalation -> support/NA. - Contains
retention|governance|pii|backup -> governance/NA. - Else general/NA.

Retrieval output shape retrieve_docs(query, docs, top_k) returns
list[RetrievedDoc], where each item has: - doc: KnowledgeDoc with
{source, title, text} - score: float

Lexical score formula in knowledge_base.py: - query_terms = _tokenize(query)
- doc_terms = _tokenize(doc.text) - score = len(query_terms &
doc_terms) / max(1, len(query_terms)) - Sort descending and trun-
cate to top_k.

3.4 Flow D: Planning and reviewing (LLM + fallback)

Planner request shape (AsyncOllamaGateway.chat) Planner sends:
- model: from settings.planner_model - messages: list of {"role":
"system"|"user", "content": "..."} - temperature, max_tokens,
timeout_seconds

Planner success output: ChatResult. Planner fallback conditions: - exception
in API call - empty result.text.strip()

Fallback output text is deterministic and includes: - route context
(intent/severity) - top retrieved source and text (if any)

Reviewer output shape ReviewerAgent.review(...) returns ChatResult
where: - text is planner draft + appended reviewer checks - fallback_used is
False unless draft was empty - token/duration metadata copied from planner
ChatResult

3.5 Flow E: Evaluation loop and metrics

Source: src/multi_agent_system/eval/evaluator.py

1. load_eval_tasks(path) parses JSON to list[TaskExample].
2. For each task, coordinator runs with trace ID: eval-{idx:03d}-{task.task_id}.
3. Compute:
 - baseline_keyword_recall
 - final_keyword_recall
 - keyword_gain
 - retrieval_hit (expected_source found in retrieved sources)

- source_cited_in_final (expected_source string appears in final answer)

4. Aggregate EvalSummary means/rates.

Task JSON shape (data/eval/tasks.json):

```
{
  "task_id": "t1",
  "question": "When should emergency rollback be triggered for API incidents?",
  "reference_answer": "Rollback is required when p95 latency exceeds 1200 ms for 10 consecutive",
  "expected_source": "incident_triage.md",
  "required_keywords": ["1200", "10", "minutes", "rollback"]
}
```

Prediction row shape (EvalPrediction-> CSV columns in artifacts/evals/predictions.csv):

```
- task_id, question, expected_source - baseline_answer, final_answer
- baseline_keyword_recall, final_keyword_recall, keyword_gain
- retrieval_hit, source_cited_in_final - planner_fallback_used,
reviewer_fallback_used - total_latency_ms
```

Summary shape (EvalSummary, persisted in artifacts/evals/summary.json):

```
{
  "n_tasks": 6,
  "baseline_keyword_recall_mean": 0.0,
  "multi_agent_keyword_recall_mean": 0.8333333333333334,
  "keyword_recall_gain": 0.8333333333333334,
  "retrieval_hit_rate": 1.0,
  "source_citation_rate": 1.0,
  "planner_fallback_rate": 1.0,
  "reviewer_fallback_rate": 0.0,
  "avg_total_latency_ms": 13027.571833333333
}
```

3.6 Flow F: Telemetry and reporting

Span record shape (TraceSpanRecord) Each line in artifacts/telemetry/traces.jsonl is:

```
{
  "trace_id": "eval-001-t1",
  "span_name": "route",
  "status": "ok",
  "start_time_utc": "2026-06-12T11:39:29.927+00:00",
  "end_time_utc": "2026-06-12T11:39:29.927+00:00",
  "latency_ms": 0.02,
  "attributes": {"question_chars": 62}
}
```


Telemetry summary shape (summarize_traces) Output in artifacts/telemetry/summary.json:

- top-level: generated_at_utc, trace_file, n_spans, n_unique_traces,
by_span - each by_span row: span_name, count, error_count, latency_ms_mean,
latency_ms_p50, latency_ms_p95, latency_ms_max

Report rendering render_report(...) in reporting.py uses Jinja2 template and writes: - artifacts/reports/multi_agent_system_report.md

3.7 Flow G: Full run wrapper (run_all)

run_all(settings) does: 1. Define 3 demo questions. 2. Remove existing trace file if present. 3. Run demos (run_demo) and evaluation (run_evaluation(reset_traces=False)). 4. Persist artifacts/run_summary.json with paths and summary payload.

run_all payload shape:

```
{
  "planner_model": "phi3.5:3.8b",
  "reviewer_model": "functiongemma:270m",
  "demo_runs_path": ".../artifacts/runs/demo_runs.json",
  "demo_questions": ["...", "...", "..."],
  "demo_count": 3,
  "evaluation": {
    "summary": {"n_tasks": 6, "...": "..."},
    "predictions_path": ".../artifacts/evals/predictions.csv",
    "summary_path": ".../artifacts/evals/summary.json",
    "report_path": ".../artifacts/reports/multi_agent_system_report.md",
    "trace_path": ".../artifacts/telemetry/traces.jsonl",
    "telemetry_summary_path": ".../artifacts/telemetry/summary.json"
  },
  "run_summary_path": ".../artifacts/run_summary.json"
}
```

Module 4: Setup & Run Guide

4.1 Prerequisites

From README.md and pyproject.toml: - Linux/macOS shell environment -
Python 3.12.10 - uv for environment and package management - Local Ollama
runtime and pulled models

4.2 Installation and environment setup

Typical sequence:

```
git clone https://github.com/pypi-ahmad/multi-agent-system-ollama.git
cd multi-agent-system-ollama
```

```
uv python pin 3.12.10
uv sync --dev
cp .env.example .env
```

4.3 Required local models

```
ollama pull phi3.5:3.8b
ollama pull functiongemma:270m
```

4.4 Environment variables and config keys

The runtime `Settings` class loads from `.env` (`env_file=".env"`, case-insensitive).

Keys shown in `.env.example`: - `OLLAMA_HOST` - `PLANNER_MODEL` - `REVIEWER_MODEL` - `SEED` - `RETRIEVAL_TOP_K` - `GENERATION_TEMPERATURE` - `GENERATION_MAX_TOKENS` - `GENERATION_TIMEOUT_SECONDS` - `KNOWLEDGE_DIR` - `EVALUATION_FILE` - `ARTIFACTS_DIR` - `Eval_DIR` - `REPORT_DIR` - `TELEMETRY_DIR` - `RUNS_DIR`

Important validation constraints from `Settings`: - `retrieval_top_k`: 1 to 10 - `generation_temperature`: 0.0 to 1.0 - `generation_max_tokens`: 32 to 2048 - `generation_timeout_seconds`: 1.0 to 60.0

4.5 Main command sequences

CLI command group: `multi-agent-system`.

Core sequences from `README.md/cli.py`:

```
uv run multi-agent-system check-models
uv run multi-agent-system run-demo
uv run multi-agent-system evaluate
uv run multi-agent-system summarize-telemetry
uv run multi-agent-system run-all
```

Script alternatives:

```
uv run python scripts/run_pipeline.py
uv run python scripts/generate_notebooks.py
uv run python scripts/execute_notebooks.py
```

4.6 Data bootstrap / migration / seeding status

- No database layer exists in this repository.
- No migrations or seed scripts are required.
- Runtime data dependencies are file-based:
 - Knowledge documents in `data/knowledge/`
 - Evaluation tasks in `data/eval/tasks.json`

4.7 CI quality gates

From `.github/workflows/ci.yml`: `- uv sync --dev - uv run ruff check .`
`- uv run mypy src - uv run pytest`

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

1. **Read project overview and setup contract**
 - Files: `README.md`, `pyproject.toml`, `.env.example`
 - Goal: understand runtime prerequisites and entrypoint wiring.
2. **Understand typed contracts first**
 - File: `src/multi_agent_system/schemas.py`
 - Goal: learn data shapes before reading flow logic.
3. **Trace orchestration end-to-end**
 - Files: `src/multi_agent_system/cli.py`, `src/multi_agent_system/pipeline.py`, `src/multi_agent_system/orchestration/coordinator.py`
 - Goal: map command -> pipeline -> agent stages -> artifacts.
4. **Deep dive each agent**
 - Files: `src/multi_agent_system/agents/router.py`, `retriever.py`, `planner.py`, `reviewer.py`
 - Goal: understand specialization and fallback behavior.
5. **Study retrieval data and tasks**
 - Files: `src/multi_agent_system/tools/knowledge_base.py`, `data/knowledge/*.md`, `data/eval/tasks.json`
 - Goal: connect corpus facts to evaluation expectations.
6. **Study metrics and observability**
 - Files: `src/multi_agent_system/eval/evaluator.py`, `src/multi_agent_system/telemetry/tracer.py`, `src/multi_agent_system/reporting/reporting.py`
 - Goal: understand how quality and latency are measured and reported.
7. **Use tests as behavior specs**
 - Files: `tests/*.py`
 - Goal: verify your mental model against explicit assertions.

5.2 Practice exercises (with model solution outlines)

1. **Exercise:** Explain the exact control flow of `run-all` from CLI command to final artifacts.
 - Read: `cli.py`, `pipeline.py`, `reporting.py`, `telemetry/tracer.py`
2. **Exercise:** For question "We have outage and need rollback", what route intent and severity are returned, and why?
 - Read: `agents/router.py`
3. **Exercise:** Reproduce retrieval scoring for query "When rollback at 1200 ms?" using docs in `tests/test_knowledge_retrieval.py`.
 - Read: `tools/knowledge_base.py`, `tests/test_knowledge_retrieval.py`

4. **Exercise:** List all fallback paths in planner/reviewer and the exact conditions that trigger each.
 - Read: `agents/planner.py`, `agents/reviewer.py`
5. **Exercise:** Write the exact `TaskExample` JSON schema and explain how `evaluate()` converts one task into one prediction row.
 - Read: `schemas.py`, `eval/evaluator.py`, `data/eval/tasks.json`
6. **Exercise:** Identify span names recorded per run and explain where each span starts/ends.
 - Read: `orchestration/coordinator.py`, `telemetry/tracer.py`
7. **Exercise:** Explain how `Settings.resolve()` and `resolved_*` properties prevent path ambiguity.
 - Read: `config.py`
8. **Exercise:** Compare `baseline_answer` vs `final_answer` fields in `artifacts/evals/predictions.csv` and explain why gains can occur even when planner fallback is `True`.
 - Read: `artifacts/evals/predictions.csv`, `agents/planner.py`, `agents/reviewer.py`, `eval/evaluator.py`

5.3 Model answers / solution outlines

1. **Run-all flow:** `run_all_cmd()` -> `run_all(settings)` -> clear traces -> `run_demo()` -> `run_evaluation(reset_traces=False)` -> save predictions/summary/telemetry/report -> save `run_summary.json`.
2. **Outage + rollback route:** intent incident, severity S1, reason incident keywords matched (because outage/rollback branch forces S1).
3. **Retrieval score:** query terms overlap with doc A (rollback, 1200, ms) is highest; `top_k=1` returns source a.md.
4. **Fallback paths:** planner fallback on exception/empty text; baseline fallback on exception/empty baseline; reviewer fallback on empty draft text.
5. **Task -> prediction row:** run coordinator, compute keyword recalls and gain, set retrieval/source-citation booleans, copy fallback flags and latency into `EvalPrediction`.
6. **Span names:** route, retrieve, baseline_answer, plan_answer, review_answer; each wraps the corresponding stage in coordinator.
7. **Path resolution:** relative paths are resolved against `project_root`; all output directories are created by `ensure_dirs()`, reducing runtime path errors.
8. **Why gain with planner fallback:** fallback text still includes retrieved source text and can contain required keywords, while baseline often returns "I do not know..."

Learner Verification Checklist

Use this checklist after studying:

- ☐ Can I explain how `multi-agent-system run-all` travels through `cli.py` -> `pipeline.py` -> `coordinator.py`?
- ☐ Can I describe every field in `AgentRunResult`, `EvalPrediction`, and `EvalSummary`?
- ☐ Can I explain router keyword rules and resulting `intent/severity` outputs?
- ☐ Can I manually compute lexical overlap retrieval score for one query-document pair?
- ☐ Can I identify all planner/reviewer fallback conditions and outputs?
- ☐ Can I explain how telemetry spans are written and summarized (mean/p50/p95/max)?
- ☐ Can I map each artifact file to the function that writes it?
- ☐ Can I modify `.env` settings confidently and predict their effect on runtime behavior?
- ☐ Can I explain why this design is robust even when LLM calls fail?